

CAS4160 Homework 1: Imitation Learning

[Your name]
[Your student ID]

1 Introduction

이 과제의 목표는 (1) 연세 AI Teaching Cluster (VESSL AI), (2) 강화학습 환경(Reinforcement Learning Environments), (3) 모방학습 알고리즘(Imitation Learning Algorithms) – 특히 **behavioral cloning (BC)**과 **DAgger** – 에 익숙해지는 것입니다. 여기 과제 보고서 템플릿의 Overleaf 링크가 있습니다.

2 (Optional) LaTeX

LaTeX에 익숙하지 않다면, Overleaf 튜토리얼인 “Learn LaTeX in 30 minutes.”¹을 꼭 한 번 훑어보기를 강력히 권장합니다.

더 자세한 안내가 필요하다면 Overleaf의 공식 문서를 참고하세요.²

이번 과제에서는 Overleaf를 다음과 같이 사용해야 합니다:

1. Overleaf 계정을 생성합니다.
2. 화면 왼쪽 상단의 “Menu”를 선택한 뒤 “Copy Project”를 클릭하여 프로젝트를 복사합니다. 이렇게 하면 main.tex 파일을 자유롭게 수정할 수 있습니다.
3. `\author{...}` 부분의 플레이스홀더를 본인의 이름과 학번으로 바꿉니다 (37번째 줄과 38번째 줄).
4. 내용을 수정한 후에는 오른쪽 PDF 패널에서 업데이트된 결과를 확인하기 위해 “Recompile” 버튼을 누릅니다.
5. 제목 아래에 본인의 이름과 학번이 올바르게 표시되는지 확인합니다.
6. main.tex 파일을 처음부터 끝까지 읽어보고, 이것저것 직접 수정하면서 돌아보세요!

3 (Optional) VESSL AI

자신의 GPU가 있다면 이 섹션은 건너뛸 수 있습니다. Google Colab을 이용하는 것도 가능합니다.

이번 과제에 GPU가 필요하다면, 연세대학교 AI학과 강의용 GPU 클러스터인 VESSL AI (AI-LEC-1 그룹) 를 제공합니다.

여기서는 RTX 3090 GPU를 월 300시간 동안 사용할 수 있습니다.

여기 workspace(즉, 사용자에게 GPU 1장이 할당된 가상 머신)를 실행하고 사용하는 방법에 대한 안내가 있습니다.

주의사항:

- 사용하지 않을 때는 반드시 workspace를 정지(stop) 시켜 컴퓨팅 자원을 절약해 주세요.
- VESSL Workspace는 최대 72시간의 런타임만 할당됩니다. 이 시간을 초과하면 workspace가 자동으로 정지됩니다. 사용 가능한 시간을 관리하기 위해 사용량을 주기적으로 확인하고, 자동 정지된 경우 workspace를 다시 시작하세요.

¹https://www.overleaf.com/learn/latex/Learn_LaTeX_in_30_minutes

²<https://www.overleaf.com/learn>

4 Play with Reinforcement Learning Environments

과제 자료 안에 포함된 튜토리얼(GymTutorial.ipynb)을 제공합니다. 이 튜토리얼은 강화학습(RL) 환경에 익숙해지는 데 도움이 되도록 준비되었습니다. gym 환경과 어떻게 상호작용하는지 이해하기 위해 이 튜토리얼을 검토하고 직접 실행해 보시기를 강력히 권장합니다. 튜토리얼을 완료한 후, gym 환경의 기본 원리를 이해했는지 확인하기 위해 아래 질문에 답변해 주세요.

4.1 Questions:

1. `step()` 함수의 역할을 설명하세요. 어떤 종류의 정보를 반환하나요?
Answer: Include your answer here.
2. gym 환경에서 `reset()` 함수의 역할을 설명하세요. 이 함수의 반환값은 무엇인가요?
Answer: Include your answer here.
3. 어떤 gym 환경이 discrete action space를 가지는지, 아니면 continuous action space를 가지는지 어떻게 알 수 있나요?
Answer: Include your answer here.

5 Behavioral Cloning

5.1 Code Structure Overview

5.1.1 Main Script: `run_hw1.py`

- 행동 복제(Behavioral Cloning) 실험을 실행하기 위한 진입점(entry point)
- 명령줄 인자(command-line arguments)를 파싱하고 실험 파라미터를 설정합니다.
- 지정된 파일에서 전문가 정책(expert policy)을 로드합니다.
- `BCTrainer`를 초기화하고 학습(training)을 실행합니다.
- 로깅(logging)과 실험 출력 결과를 저장하는 작업을 처리합니다.

5.1.2 Trainer: `bc_trainer.py`

- Defines `BCTrainer`, training loops 관리, data collection, 그리고 logging을 담당하는 클래스/모듈
- Key methods:
 - `run_training_loop()`: 여러 번의 학습 반복(iteration)을 실행합니다.
 - `collect_training_trajectories()`: 현재 정책(policy)을 사용해 데이터를 수집(trajecory 모으기)합니다.
 - `train_agent()`: 샘플링된 trajectory들을 이용해 에이전트를 업데이트(학습)합니다.
 - `do_relabel_with_expert()`: expert policy을 이용해 행동(action)을 수정합니다 (DAgger에서 사용).
 - `perform_logging()`: 학습 및 평가 통계(statistics)를 로깅합니다.

5.1.3 Behavioral Cloning Agent: `bc_agent.py`

- Implements `BCAgent`, 행동 복제(Behavioral Cloning)를 통해 학습된 에이전트를 나타내는 클래스/모듈

- Key attributes:
 - actor: `MLPPolicySL`을 사용하여 관측(observations)을 행동(actions)으로 매핑합니다.
 - replay buffer: 과거 trajectory(궤적)들을 저장합니다.
- Key methods:
 - train(): 지도 학습(supervised learning)을 이용해 정책(policy)을 업데이트합니다.
 - add to replay buffer(): 새로운 rollout(에피소드 실행 결과)을 저장합니다.
 - sample(): replay buffer에서 학습에 사용할 데이터를 샘플링(추출)합니다.

5.1.4 Policy Network: MLP policy.py

- Implements `MLPPolicySL`, a feedforward neural network for behavioral cloning.
- Key methods:
 - get action(): 주어진 관측(observation)을 입력으로 받아 행동(action)을 계산(추론)합니다.
 - forward(): 네트워크의 순전파(forward pass)를 정의합니다.
 - update(): 지도 학습 손실 함수(supervised loss function)를 사용하여 정책(policy)을 학습(업데이트)합니다.

5.1.5 Replay Buffer: replay buffer.py

- 전이(transition)를 저장하고 샘플링하기 위한 간단한 경험 재생 버퍼(experience replay buffer)를 구현합니다.
- Key methods:
 - add rollouts(): 새로운 trajectory(궤적)들을 버퍼에 추가합니다.
 - sample random data(): 경험(experiences)의 무작위 배치(batch)를 반환합니다.

5.1.6 Utility Functions: utils.py

- 환경과의 상호작용을 위한 helper 함수들을 포함합니다.
- Key methods:
 - sample trajectory(): 단일 에피소드의 경험을 수집합니다.
 - sample trajectories(): 여러 번의 rollout(에피소드 실행)을 모읍니다.
 - sample n-trajectories(): 고정된 수의 trajectory(궤적)를 수집합니다.
 - get trajlength(): trajectory의 길이를 계산합니다.

5.1.7 PyTorch Helper Functions: pytorch_util.py

- 신경망(neural network) 구성과 텐서(tensor) 처리를 위한 유틸리티 함수들을 제공합니다.
- Key methods:
 - build mlp(): 다층 퍼셉트론(multi-layer perceptron)을 구성(생성)합니다.
 - from numpy(), to numpy(): NumPy 배열과 PyTorch 텐서 사이를 변환합니다.

5.2 Implement Behavioral Cloning

당신의 임무는 코드에서 TODO로 표시된 부분들을 채우는 것입니다. 특히 다음 파일들을 확인하세요:

- `cas4160/infrastructure/bc_trainer.py`(단, `do_relabel_with_expert` 함수는 다음 섹션인 DAgger를 위한 것이므로 이번에는 구현하지 않습니다.)
- `cas4160/policies/MLP_policy.py`
- `cas4160/infrastructure/replay_buffer.py`
- `cas4160/infrastructure/utils.py`
- `cas4160/infrastructure/pytorch_util.py`

권장 구현 순서는 다음과 같습니다:

1. `pytorch_util.py`에서 `build_mlp()`를 먼저 구현하세요.(정책 네트워크에서 사용되기 때문입니다.)
2. `MLP_policy.py`에서 정책 네트워크를 구현하세요.(에이전트에서 사용됩니다.)
3. `utils.py`에서 유틸리티 함수들을 구현하세요.(기본적인 trajectory 샘플링 함수들을 제공합니다.)
4. `replay_buffer.py`에서 replay buffer를 구현하세요.(에이전트 학습에 필요합니다.)
5. `bc_agent.py`에서 behavioral cloning 에이전트를 구현하세요.
6. `bc_trainer.py`에서 trainer를 구현하세요.
7. 위의 모든 구현을 완료했다면, `run_hw1.py`를 실행하여 전체 파이프라인을 연결하고 학습을 시작할 수 있습니다.

Behavioral Cloning (BC)을 실행하고, 두 가지 태스크에 대한 결과를 보고하세요: (1) Ant-v4 환경 → behavioral cloning 에이전트가 전문가(expert) 성능의 최소 30% 이상을 달성해야 합니다. (2) Walker2d-v4, HalfCheetah-v4, Hopper-v4 중 하나를 선택 → 해당 환경에서도 전문가 데이터가 제공되므로 BC를 수행합니다.

전문가 정책의 성능은 로그 출력에서 Initial DataCollection AverageReturn 항목을 통해 확인할 수 있습니다.

위의 TODO들을 모두 구현한 후에는 다음과 같은 방법으로 Ant 태스크에 대한 BC 정책(behavioral cloning policy)을 학습할 수 있습니다:

```
python cas4160/scripts/run_hw1.py \
  --expert_policy_file cas4160/policies/experts/Ant.pkl \
  --env_name Ant-v4 --exp_name bc_ant --n_iter 1 \
  --expert_data cas4160/expert_data/expert_data_Ant-v4.pkl \
  --video_log_freq -1
```

실행이 성공적으로 완료되면, TensorBoard 로그 데이터는 다음 경로에서 확인할 수 있습니다:

`hw1-starter code/data/q1 [--exp name] [--env name] [current time]/`.

결과를 보고할 때는 다음 사항을 준수해 주세요: 여러 번의 rollout(에피소드 실행)을 통해 얻은 정책의 return(누적 보상)에 대한 평균(mean)과 표준편차(standard deviation)를 표(table) 형식으로 보고하세요. 사용한 태스크(task)가 무엇인지 명시하세요. 잘 작동하는 경우와 그렇지 않은 경우를 비교할 때는, 네트워크 크기(network size), 사용한 데이터 양(amount of data), 학습 반복 횟수(number of training iterations) 등을 동일하게 맞춰 공정한 비교(fair comparison)를 해야 합니다. 위 사항들(그리고 적절하다고 판단되는 기타 세부사항)을 표의 캡션(caption)에 함께 기재해 주세요.

참고: “평균과 표준편차를 보고하라(report the mean and standard deviation)”는 말은, 평가 시 eval batch size를 에피소드 길이(ep_len)보다 충분히 크게 설정하여, 학습된 정책의 성능을 평가할 때 여러 rollout(에피소드)를 수집한다는 의미입니다. 예를 들어, ep_len이 1000이고 eval_batch_size가 5000이라면, 대략 5개의 에피소드(일부 에피소드가 조기에 종료되면 더 많아질 수 있음)를 수집하게 되며, 로그에 기록되는 Eval AverageReturn과 관련된 값은 이러한 여러 에피소드의 평균과 변동성을 반영하게 됩니다.

`Eval StdReturn`은 위 5개의 rollout(에피소드)에 대한 정책의 평균과 표준편차를 나타냅니다. 이 파라미터들도 반드시 표의 캡션(caption)에 포함시켜 주세요.

참고: 정책 rollout의 비디오를 생성하려면 “`--video_log_freq -1`” 플래그를 제거하면 됩니다. 단, 이 경우 실행 속도가 느려지므로 디버깅 중에는 해당 플래그를 유지하는 것이 좋습니다.

5.2.1 BC Results

Fill in the values in the template table below.

Table 1: **Your caption goes here. Please include training details here.**

Environment	Performance (Mean Return $\pm$ Std)
Ant-v4	-1 $\pm$ -1
[Env you've chosen]	-1 $\pm$ -1

5.3 Rendering and Evaluating Rollouts

렌더링 환경에 익숙해지기 위해, 학습된 정책의 평가 rollout(에피소드 실행) 스크린샷을 반드시 포함해야 합니다. TensorBoard를 사용 중이라면, `eval rollouts` 섹션에서 확인할 수 있습니다. 학습 진행 상황을 분석할 때는 단순히 숫자만 보는 것보다 렌더링된 비디오를 직접 보고 확인하는 것이 매우 중요합니다!

5.3.1 Environment Rendering

Figure:



Figure 1: 여기에 여러분의 플롯(plot)을 삽입하세요. 그리고 캡션(caption)을 추가해 주세요. PDF 형식이 가장 권장되지만, PNG나 JPG 형식도 허용됩니다.

5.4 Hyperparameter Tuning of Behavioral Cloning

하나의 하이퍼파라미터 세트를 실험하여 behavioral cloning 에이전트의 성능에 어떤 영향을 미치는지 확인하세요. 예를 들어 다음 중 하나를 선택할 수 있습니다: 학습 스텝 수 (amount of training steps) 제공되는 전문가 데이터의 양 (amount of expert data provided) 또는 여러분이 직접 생각했던 다른 하이퍼파라미터 이전 질문에서 사용했던 태스크 중 하나를 선택하여, 해당 하이퍼파라미터의 값에 따라 BC 에이전트의 성능이 어떻게 변화하는지를 보여주는 그래프를 제시하세요. 또한 다음 사항을 명시해 주세요: 선택한 하이퍼파라미터가 무엇인지 그 하이퍼파라미터를 선택한 간단한 이유(rationale)

선택한 하이퍼파라미터에 대해 최소 4가지 다른 설정을 포함해야 합니다. 여기에는 이전 부분에서 사용했던 기본 설정(default setting)도 반드시 포함되어야 합니다.

참고: 명령줄 인자(command line arguments)를 통해 지정할 수 있는 몇 가지 기본 하이퍼파라미터가 있습니다. 아래에 나열된 하이퍼파라미터 중 하나를 선택하는 것을 추천합니다:

- 정책 학습을 위한 gradient step 수 (--num_agent_train_steps_per_iter, 기본값: 1000)
- 학습 데이터의 양 (--batch_size, 기본값: 1000)
- 학습 배치 크기 (--train_batch_size, 기본값: 100)
- 정책 신경망의 깊이 (레이어 수) (--n_layers, 기본값: 2)
- 정책 신경망의 너비 (유닛 수) (--size, 기본값: 64)
- 지도 학습을 위한 학습률 (--learning_rate, 기본값: 5e-3)

스크립트를 실행할 때 명령줄에서 해당 하이퍼파라미터를 지정할 수 있습니다. 예를 들어, 다음과 같이 명령어를 실행하면 정책을 500번의 gradient step 동안 학습할 수 있습니다:

```
python cas4160/scripts/run_hw1.py \  
  --num_agent_train_steps_per_iter 500 \  
  --some other arguments...,
```

Note: matplotlib을 사용하여 플롯(plot)을 그려주세요. matplotlib에 익숙하지 않다면, 공식 튜토리얼을 참고할 수 있습니다..

5.4.1 Hyperparameter Tuning Results

Hyperparameter: 선택한 하이퍼파라미터와 테스트한 네 가지 다른 값을 작성하세요. **Plot:**

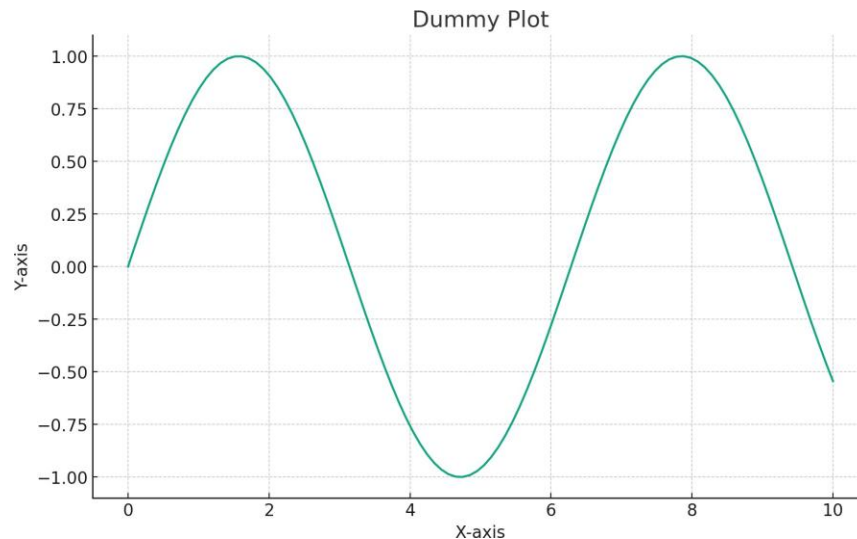


Figure 2: 여기에 여러분의 플롯(plot)을 삽입하세요. 그리고 캡션(caption)을 추가해 주세요. PDF 형식이 가장 권장되지만, PNG나 JPG 형식도 허용됩니다.

Rationale: 여기에 여러분의 rationale(이유/근거)을 작성하세요.

6 DAgger

6.1 Implement DAgger

이제 여러분의 임무는 DAgger 알고리즘을 구현하는 것입니다. 만약 BC 부분을 올바르게 구현했다면, 다음 파일에 있는 TODO만 구현하면 됩니다.

- cas4160/infrastructure/bc_trainer.py 파일 내의 `do_relabel_with_expert function`

코드에서 TODO 주석으로 지정된 모든 지시사항을 채웠다면, 아래 명령어를 사용해 DAgger를 학습시킬 수 있어야 합니다:

```
python cas4160/scripts/run_hw1.py \
--expert_policy_file cas4160/policies/experts/Ant.pkl \
--env_name Ant-v4 --exp_name dagger_ant --n_iter 10 \
--do_dagger \
--expert_data cas4160/expert_data/expert_data_Ant-v4.pkl \
--video_log_freq -1
```

6.2 Compare BC and DAgger

DAgger를 실행하고, 이전에 BC로 테스트했던 두 가지 태스크(Ant + 다른 하나의 환경)에 대한 결과를 보고하세요. 결과는 학습 곡선(learning curve) 형태로 제시해야 합니다. x축: DAgger 반복(iteration) 수 y축: 정책의 평균 return (mean return) 캡션(caption) 에는 다음 사항을 반드시 포함하세요: 사용한 태스크(task)가 무엇인지 네트워크 구조(network architecture), 사용한 데이터 양(amount of data) 등에 대한 세부 사항 (이전 섹션에서와 동일한 방식으로 기재)

참고: TensorBoard 로그에서 데이터를 파싱하고 그림을 그리는 예제 헬퍼 스크립트(cas4160/scripts/parse_tensorboard.py)를 사용할 수 있습니다. 아래는 그림을 output_plot.png 파일로 저장하는 예제 사용법입니다:

```
python cas4160/scripts/parse_tensorboard.py \
--input_log_files data/[replace_here_with_the_name_of_log_folder] \
--data_key "Eval_AverageReturn" \
--title "DAgger: Ant-v4" \
--x_label_name "DAgger iterations" \
--y_label_name "Mean Return" \
--output_file "output_plot.png"
```

또한 BC 정책과 전문가(expert) 정책의 성능을 수평선(horizontal line)으로 플롯(plot)하고, 표준편차(standard deviations)를 에러 바(error bars)로 표시하는 것도 고려해 볼 만합니다. 예제 파싱 스크립트는 원하는 대로 자유롭게 수정해 사용하세요.

6.2.1 DAgger Result

여기에 두 가지 태스크에 대한 플롯(plot)을 포함하세요.

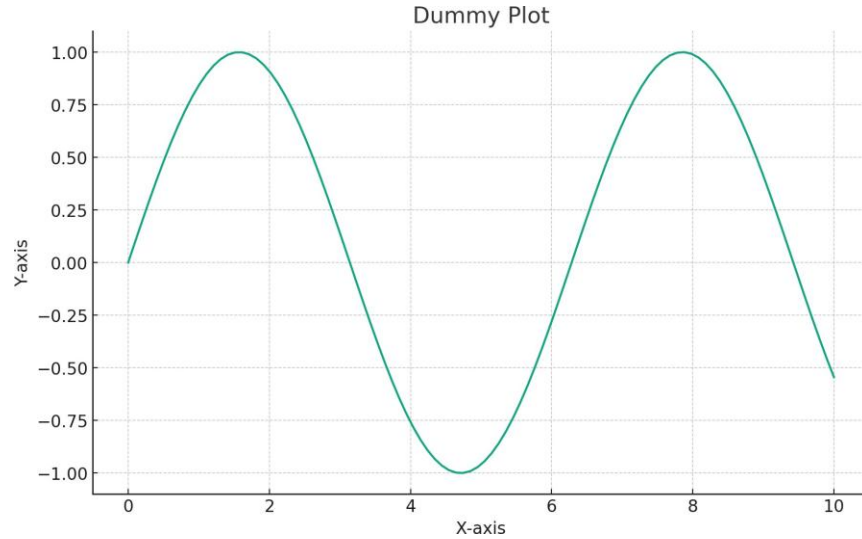


Figure 3: 여기에 여러분의 플롯(plot)을 삽입하세요. 그리고 캡션(caption)을 추가해 주세요.

6.2.2 Justification of the Result

DAgger, BC, 그리고 전문가(expert) 정책의 성능을 비교하고, 결과를 설명하세요.

7 Submission

과제를 제출할 때는 코드와 “보고서”를 하나의 zip 파일로 묶어서 제출해 주세요. 파일명은 hw1_[학번].zip 형태로 하시면 됩니다. (예: hw1_2023123456.zip) zip 파일 크기는 20MB 미만이어야 합니다. 제출 파일의 구조는 다음과 같아야 합니다:

```
hw1_[YourStudentID].zip hw1
├── [YourStudentID].pdf
├── cas4160
│   └── agents
│       ├── bc_agent.py
│       └── ...
```

Note: 비디오 파일(.mp4) 과 TensorBoard 로그는 절대 포함시키지 마세요.